Computación y Estructuras Discretas III

Andrés A. Aristizábal P.
aaaristizabal@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes

2026-1

Agenda del día

1. **Grammars and Languages**
   - Implementing a robot language with textx
   - Exercises

Agenda del día

1. **Grammars and Languages**
   - Implementing a robot language with textx
   - Exercises

Implementing a robot language with textx

What is the first step when building a DSL?

What is the first step when building a DSL?

- We should first do a domain analysis, to see what concepts do we have and what are their relationships and constraints.

Implementing a robot language with textx

What to do in this case?

Implementing a robot language with textx

What to do in this case?

- In this case we want an imperative language that should define robot movement on the imaginary grid.
- Robot should move in four base direction.
- We will call these directions up, down, left and right.
- Additionally, we shall have a robot coordinate given in x, y position.
- For simplicity, our robot can move in discrete steps. In each movement robot can move by 1 or more steps but in the same direction.
- Coordinate is given as a pair of integer numbers.
- Robot will have an initial position.
- If not given explicitly it is assumed that position is (0, 0).

Implementing a robot language with textx

What would be an example program in our robot language?

Implementing a robot language with textx

What would be an example program in our robot language?

```
begin
   initial 3, 1
   up 4
   left 9
   down
   right 1
end
```

Implementing a robot language with textx

Let's start with the grammar

```
Program:
  'begin'
    commands*=Command
  'end'
;
```

Implementing a robot language with textx

We define our Command rule

```
Command:
  InitialCommand | MoveCommand
;
```

Now our two types of commands

```
InitialCommand:
  'initial' x=INT ',' y=INT
;

MoveCommand:
  direction=Direction (steps=INT)?
;
```

Implementing a robot language with textx

We need to define our direction rule

```
Direction:
  "up"|"down"|"left"|"right"
;
```

Implementing a robot language with textx

We may have comments in our program

```
Comment:
  /\/\/.*$/
;
```

Implementing a robot language with textx

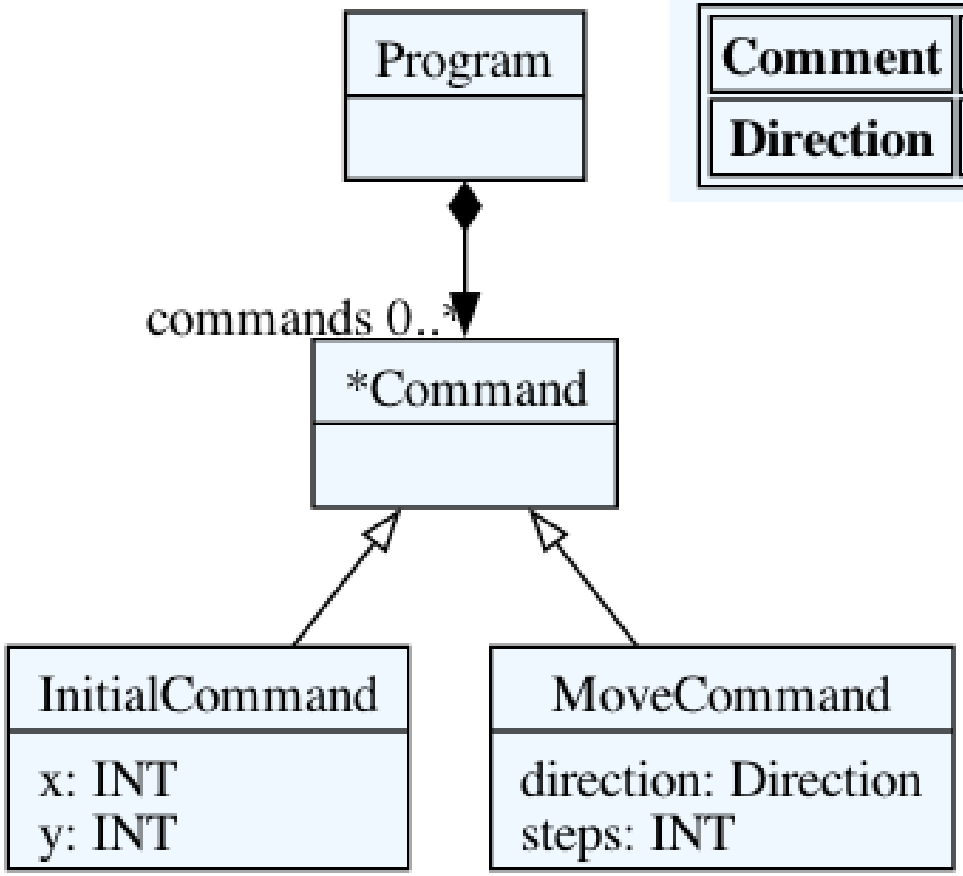
- We save our grammar in a file (robot.tx)
- In order to comprehend our language we generate a meta-model visualization.
- We run the command `!textx generate robot.tx --target dot`
- We get a .dot file which is a graph description language file.
- To export it as a png file we run this command in a cell of our Jupyter notebook

```
import pydot

(graph,) = pydot.graph_from_dot_file('robot.dot')
graph.write_png('robot.png')
```

If this doesn't work you can always export as png using `https://dreampuf.github.io/GraphvizOnline/` with the contents in the dot file.

Implementing a robot language with textx



Comment	\\.*$
Direction	up|down|left|right

Implementing a robot language with textx

We need to instantiate the meta-model:

Implementing a robot language with textx

We need to instantiate the meta-model:

- In order to parse our models we first need to construct a meta-model.
- A textX meta-model is a Python object that contains all classes that can be instantiated in our model.
- For each grammar rule a class is created.
- Additionally, the meta-model contains a parser that knows how to parse input strings.
- From parsed input (parse tree) the meta-model will create a model.

Implementing a robot language with textx

- We start our python or ipynb file with the following command to create the meta-model from the grammar description

```
from textx import metamodel_from_file
robot_mm = metamodel_from_file('robot.tx')
```

Implementing a robot language with textx

We save our example of a program in the robot language as example_robot.rbt

```
begin
   initial 3, 1
   up 4
   left 9
   down
   right 1
end
```

Implementing a robot language with textx

- Then we use the meta-model to create models from textual description:

```
robot_model = robot_mm.model_from_file('robot.rbt')
```

Implementing a robot language with textx

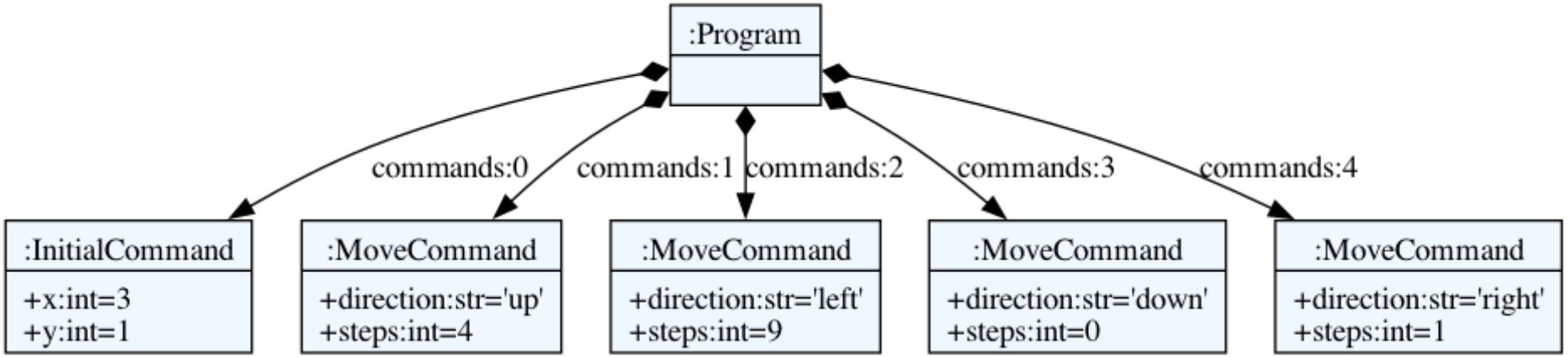
- We generate a meta-model visualization.
- We run the command `!textx generate example_robot.rbt --grammar robot.tx --target dot`
- We get a .dot file which is a graph description language file.
- To export it as a png file we run this command in a cell of our Jupyter notebook

```
import pydot

(graph,) = pydot.graph_from_dot_file('example_robot.dot')
graph.write_png('example_robot.png')
```

If this doesn't work you can always export as png using `https://dreampuf.github.io/GraphvizOnline/` with the contents in the dot file.

Implementing a robot language with textx



Interpreting the language

- We will build a simple interpreter that will start the robot from the initial position and print the position of the robot after each command.

```
class Robot(object):
    def __init__(self):
        self.x = 0
        self.y = 0
    def __str__(self):
        return f"Robot position is {self.x}, {self.y}."
```

Implementing a robot language with textx

- We add an interpret method that accepts our robot model and runs it.

```
def interpret(self, model):
    for c in model.commands:
        if c.__class__.__name__ == "InitialCommand":
            print(f"Setting position to: {c.x}, {c.y}")
            self.x = c.x
            self.y = c.y
        else:
            print(f"Going {c.direction} for {c.steps} step(s).")
            move = {
                "up": (0, 1),
                "down": (0, -1),
                "left": (-1, 0),
                "right": (1, 0)
            }[c.direction]
            self.x += c.steps * move[0]
            self.y += c.steps * move[1]
        print(self)
```

Implementing a robot language with textx

- We give our robot_model to the Robot instance

```
robot = Robot()
robot.interpret(robot_model)
```

- And we get

```
Setting position to: 3, 1
Robot position is 3, 1.
Going up for 4 step(s).
Robot position is 3, 5.
Going left for 9 step(s).
Robot position is -6, 5.
Going down for 0 step(s).
Robot position is -6, 5.
Going right for 1 step(s).
Robot position is -5, 5.
```

Implementing a robot language with textx

- It is almost correct.
- We can see that down movement is for 0 steps because we have not defined the steps for down command and haven't done anything yet to implement default of 1.
- The best way to implement default value for step is to use a so called object processor for MoveCommand.
- An Object processor is a callable that gets called whenever textX parses and instantiates an object of particular class.
- So we define our processor for MoveCommand

```
def move_command_processor(move_cmd):

  # If steps is not given, set it do default 1 value.
  if move_cmd.steps == 0:
    move_cmd.steps = 1
```

Implementing a robot language with textx

- We register this processor on the meta-model.
- After the meta-model construction we must add a line for registration.

```
robot_mm.register_obj_processors({'MoveCommand': move_command_processor})
```

- register_obj_processors accepts a dictionary keyed by class name.
- The values are callables that should handle instances of the given class.
- If you run robot interpreter again you will get output like this:

```
Setting position to: 3, 1
Robot position is 3, 1.
Going up for 4 step(s).
Robot position is 3, 5.
Going left for 9 step(s).
Robot position is -6, 5.
Going down for 1 step(s).
Robot position is -6, 4.
Going right for 1 step(s).
Robot position is -5, 4.
```

Agenda del día

1. **Grammars and Languages**
   - Implementing a robot language with textx
   - Exercises

Exercises

Exercise

Implement the Basic Robot Movement example

- *Set an initial position (initial x, y)*
- *Move in directions (up, down, left, right)*
- *Ignore comments (// ...)*

Exercises

Exercise

Extend DSL with loops that repeat commands.

- *For example: LoopCommand: 'repeat' count=INT 'times' commands*=Command 'end';*

Exercises

Exercise

Add a command to move directly to coordinates.

- *For example: GotoCommand: ‘goto’ x=INT ‘,’ y=INT;*